

XS3: A Spec-Free, Machine-Exact Triple-Graph Notation for LLM-to-LLM Communication

Vũ Văn Minh Quang

VDG Solutions

Archived: Zenodo concept DOI 10.5281/zenodo.21425813

Abstract—Large language models increasingly communicate with one another—across compaction boundaries, agent hand-offs, and tool pipelines—yet they still exchange state as free prose, which paraphrases and silently corrupts facts. We present XS3, a controlled notation in which *one sentence is exactly one labeled graph edge* (subject predicate object `.`). A document is its edge set; a small, fixed glyph vocabulary desugars mechanically into reified edges, so every well-formed document decodes to a unique triple graph. Two properties distinguish XS3. First, *spec-free comprehension*: because each token is chosen to be the strongest natural key for the concept region it activates in a pre-trained model, a language model reads XS3 correctly with no grammar, glossary, or specification supplied in context; we validate this across model families with blind cold reads. Second, *machine exactness*: a dependency-free reference decoder round-trips every construct through a canonical serialization (decode $\rightarrow$ serialize $\rightarrow$ decode is edge-exact and idempotent), passing 5000/5000 generated documents. We report an iconicity study of glyph choice that led us to replace an overloaded negation marker with the universal mathematical $\neg$, and a compaction study showing that, under a fixed budget, accumulated XS3 state degrades by *visible omission* of whole edges while equivalent prose degrades by *silent corruption*. We argue that XS3’s value is not token compression but corruption-resistant, addressable state.

Index Terms—controlled natural language, knowledge representation, triple graph, agent communication, large language models, constrained decoding, context compaction

I. INTRODUCTION

Autonomous agents built on large language models (LLMs) rarely operate in a single, unbroken context. State crosses *compaction* boundaries when a conversation is summarized to fit a window; it crosses *hand-offs* when one agent briefs another; it crosses *tool* boundaries when structured facts are serialized and re-read. At each crossing the medium is almost always free prose, and prose is lossy under paraphrase: a fact is re-worded, a qualifier is dropped, a number is smoothed, and—crucially—the reader cannot tell that a plausible-looking sentence is now wrong.

We ask a narrower question than “how should agents talk in general.” We ask: *what notation lets one model write state that another model reads back without drift, using only knowledge both models already have?* Our answer, XS3, makes three commitments:

- 1) **One sentence, one edge.** Every sentence `s p o .` is a single labeled, directed edge. A document is a *set* of edges; sentence order carries no meaning except inside an explicit ordered sequence `< >`. All metadata (time, degree, negation, deontics, provenance) is *reified*:

it becomes edges about edges, so the document is always a plain graph.

- 2) **Comprehension before parsing.** The primary correctness criterion is whether a specification-free model reads a snippet correctly, not whether a strict parser accepts it. Glyphs and core vocabulary are chosen to be the strongest *natural keys* for the regions they activate in a pre-trained model.
- 3) **Exactness through structure.** Machine determinism rides a canonical serialization of the graph, not a natural-language readback. Decoding is exact; the English render is a best-effort human view.

This paper describes the design (Sec. II–III), the exactness mechanism and its test harness (Sec. IV), a comprehension and glyph-iconicity evaluation (Sec. V), and an application to accumulated agent state and context compaction (Sec. VI), where XS3’s corruption-resistance is most visible.

II. DESIGN PRINCIPLES

A. The document is a graph

Let $G = (V, E)$ be a directed, edge-labeled multigraph. An XS3 sentence `s p o .` contributes one edge (s, p, o) . Chaining is purely syntactic sugar over the same edge set: `;` keeps the subject and switches predicate, `,` keeps subject and predicate and adds an object. Thus `an age 30; job dev. ≡ { (an, age, 30), (an, job, dev) }`. Because a document is a set, two independently written documents that denote the same graph normalize to the same canonical form and hash—an order-invariance that prose does not possess.

B. Reification keeps everything monotone

A plain triple graph is monotone and existential-positive: it can only assert that edges are present. Anything that is not a positive fact—time, strength, modality, negation, rationale—is expressed by *reifying* the target edge into a node and attaching a further edge to it. A qualifier is sugar for this: `s p o @2020` desugars to `{s p o}` at 2020; `s p o %80` to `{s p o}` deg 80. Reification is what lets a notation with negation and modality still decode to a single, plain edge set.

C. Comprehension-first and the natural-key rule

An LLM already carries, from pre-training, representations for “inverse relation,” “negation,” “prohibition,” “time,” and so on. XS3 treats a glyph as a *retrieval key* into those representations. The design rule is: each token must be the

```
#deploy a policy; owner platform.
you run tests !must.
prod ¬has feature-flags.
{tests pass} => {you tag release !must}.
```

Fig. 1. A small XS3 document. It decodes to six edges: two assert facts about #deploy; !must reifies into its edge as [you run tests] mark must; ¬has emits *only* the negation meta-edge [prod has feature-flags] mark not and never the positive triple; and the rule => relates two subgraph nodes. Order is irrelevant: any permutation denotes the same graph.

strongest natural key for the region it names. A token that is not a natural key *sinks*—the model reads past it. We found this empirically (Sec. V): a degree-zero marker pressed into service as negation was read as “degree zero,” not “not,” and had to be replaced.

A corollary is a boundary on what XS3 should invent. Structural glyphs are drawn from conventions every model has seen: mathematics ($\neg$, $\Rightarrow$, $=$, $\in$), code ($?$, $\#$, $\{\}$, $<>$), and ordinary English content words. Domain vocabulary is open Unicode—an author may write predicates and entities in any language—so XS3 is not English-only; only a small closed set of logical/structural keywords is fixed.

III. THE LANGUAGE

A. Glyphs

The structural glyphs are: $\cdot$ (end sentence), $;$ and $,$ (chaining), $\#x$ (anchor: same name, same node), $?x$ (variable, legal only inside $\{\}$), $\wedge p$ (inverse predicate), $\{ \}$ (a subgraph as one node—a world, belief, plan, or rule body), $[s\ p\ o]$ (an edge *mention*, referenced without assertion), $< >$ (the only ordered construct), $" \dots "$ (opaque string), and $//$ (comment). Three attention-anchor qualifiers ride an asserted edge—@ time, % degree, and ! a salient operator marker for engine pragmas (!must, !physics) and prohibition (!X).

B. Negation as a universal prefix

Logical negation is written with the mathematical $\neg$ as a prefix on *one* predicate ($s\ \neg p\ o$) or on *one* edge mention ($\neg[s\ p\ o]$). It desugars to a single meta-edge stating that the edge does not hold, and the positive triple is never emitted:

$an\ \neg love\ bao \rightarrow \{([an\ love\ bao],\ mark,\ not)\}.$

Negation is deliberately restricted to a single edge. A negated conjunction $\neg\{A;\ B;\ C\}$ is forbidden because, by De Morgan’s law, it denotes $\neg A \vee \neg B \vee \neg C$; disjunction has no image in a monotone edge set and would break the graph invariant. To deny several edges, one negates each.

C. Self-description

From its second line onward the language card is written in XS3 and defines XS3; each line is simultaneously a rule and a live example of itself. This is possible precisely because the notation is spec-free to read: a model can consume the self-definition without a prior grammar.

TABLE I
QUALIFIER AND NEGATION DESUGARING; EACH SURFACE FORM IS A GRAPH EQUATION.

Surface form	Reified edge(s)
$s\ p\ o\ @t$	$\{s\ p\ o\}$ at t
$s\ p\ o\ \%n$	$\{s\ p\ o\}$ deg n
$s\ p\ o\ !m$	$[s\ p\ o]$ mark m
$s\ \neg p\ o$	$[s\ p\ o]$ mark not (positive triple absent)

TABLE II
SUMMARY OF THE NINE EMPIRICAL STUDIES (§IV–§VI).

#	Study and headline result
S1	Spec-free cold comprehension: models recover steps, rules, and deontics from XS3 with no grammar in context
S2	Negation iconicity ($\%0 \rightarrow !not \rightarrow \neg$): $\neg$ read as negation in 4/4 mixed tests; ! polysemy removed
S3	Glyph audit: $\sim$ weak (model self-reports uncertainty), % and $\neg$ strong
S4	Deontic verb-anchoring: a bare-noun marker leaks polarity; a verb-anchored clause reads correctly
S5	Self-reference: the prior marker collides in metalanguage; a // comment or explicit $\neg$ reads correctly
S6	Structural round-trip: 5000/5000, English rate 1.0
S7	Budgeted compaction: XS3 13/18 exact (2 wrong) vs. prose 2/18 exact (10 silently wrong)
S8	Edge-aging prototype: 10/10 pinned exact; 73% smaller than un-aged; bounded live set
S9	Salience selection: LLM keeps 6/6 critical facts; a recency policy 0/6

IV. MACHINE EXACTNESS

Human readback (graph $\rightarrow$ English) is inherently lossy and is treated as non-normative. Machine exactness instead rides a *structural* canonical form. The reference decoder E_0 implements three operations: *decode* (XS3 $\rightarrow$ edge set), *serialize* (edge set $\rightarrow$ canonical XS3), and a best-effort *readback* (edge set $\rightarrow$ English). The correctness contract is, for every document d ,

$$\text{decode}(\text{serialize}(\text{decode}(d))) = \text{decode}(d),$$

together with idempotence of $\text{serialize} \circ \text{decode}$. A property test generates random documents spanning all constructs (chaining, anchors, sequences, rules, reified qualifiers, negation, deontics) and checks edge-set equality and idempotence. The current harness passes 5000/5000 documents with zero parse, round-trip, or idempotence failures; the informational English-readback rate is 1.0 on the same corpus. Because negation and other qualifiers desugar to ordinary meta-edges, adding the $\neg$ prefix required no change to the edge-equality contract: a document written with $\neg$ and one written with the legacy marker denote the same edge set.

V. COMPREHENSION AND ICONICITY

We evaluate the comprehension-first thesis with *blind cold reads*: a model is given only an XS3 snippet and neutral

questions, with no card, glossary, or grammar in context, and asked to reconstruct the facts. Across model families and tiers, models recover ordered steps, conditional rules, reified qualifiers, and deontics from XS3 alone. This is a claim about semantics, not surface: the operators map onto textbook graph, set, and relational-algebra notions (converse $\hat{}$, quotient $=$, closure $\Rightarrow$, membership $\in$) that every model has seen.

A. An iconicity study of negation

The natural-key rule (Sec. II-C) is not a priori; it is forced by data. We iterated the negation marker three times.

- **Degree-zero (%0).** Read as “degree zero,” never as negation—the marker sank. Rejected.
- **Salient word (!not).** Read correctly in *action* clauses, but it overloaded the $!$ operator: with negation living on $!$, models intermittently re-read $!$ must as “not must” (optional). It also failed under self-reference: a clause *about* the glyph (%0 means negation !not) was read as “%0 is negation.”
- **Universal $\neg$.** A prefix drawn from mathematics. In blind mixed-modality tests—documents interleaving $\neg$ with $!$ must and $!$ X—every model classified $\neg$ clauses as negated facts, $!$ must as mandatory, and explicitly reported that “ $!$ is not negation.” The polysemy vanished because $\neg$ carries its own strong prior and no longer leans on $!$.

A control finding sharpened the rule: a superficially similar bracketed marker that was *not* a standard symbol reintroduced the overload, whereas $\neg$ did not—confirming that the win comes from *universality of the convention*, not from separating negation *per se*. A parallel audit retired an affect qualifier whose glyph a model self-reported as “inferring from pattern, not certain,” in favor of an ordinary predicate.

B. Writing rules

Two writing rules emerged from the same method. Deontic markers must bind a *verb-anchored* clause (you force dom $!$ X), never a bare noun, which leaks its polarity. And self-referential statements about XS3’s own glyphs are commentary, not domain facts: they are best written as a // comment or plain prose, or, if they must be queryable, with an explicit $\neg$ over a quoted mention.

VI. APPLICATION: ACCUMULATED STATE AND COMPACTION

The sharpest use of XS3 is as *running state* that survives repeated re-summarization. Here the correct operation is not “rewrite more concisely” but *stack and dedup*: copy existing edges verbatim, append new ones, and drop only exact duplicates. Because edges are atomic and addressable, an old fact either survives byte-identical or is dropped as a whole edge; it is never paraphrased into a subtly wrong sentence.

TABLE III
FACT RETENTION AFTER BUDGETED COMPACTION (18 FACTS).

State medium	Fully retained	Lost	Wrong (silent)
Prose	2	6	10
XS3 edges	13	3	2

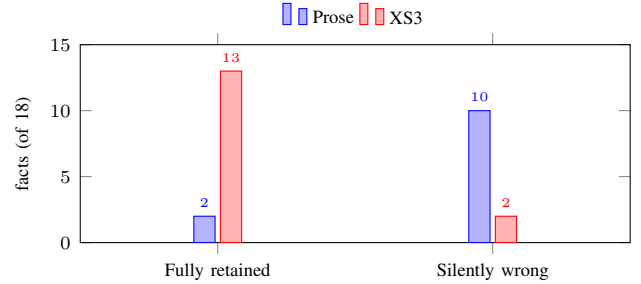


Fig. 2. Budgeted compaction (Study S7). Prose retains few facts and silently corrupts many; XS3 retains most and, when it sheds, omits whole edges rather than corrupting what it keeps.

A. A budgeted-compaction experiment

We accumulated eighteen checkable facts (DOIs, commit hashes, glyph rules, counts) into a running state under a fixed token budget that forced roughly half to be shed, comparing two arms with identical instructions and facts: one carrying state as prose, one as XS3 edges. A blind grader then scored each of the eighteen facts in each arm as *fully retained*, *lost*, or *wrong* (stated but corrupted). Table III summarizes the outcome.

The counts matter less than the *failure mode*. Prose failed by silent corruption: “desugars to [s p o] mark not” became “negates [s p o]”; “read correctly in *blind* tests” became “verified”; a rate qualifier was dropped while its fact was kept. Ten facts read plausibly and were wrong. XS3 failed by omission: it dropped whole edges (which a reader can detect as gaps), and what it retained stayed exact. For state that must be trusted downstream, a missing fact is recoverable while a silently wrong fact is not.

We note two honest limits. First, this is a fidelity result, not a compression result: on already-terse fact lists XS3 is not smaller than prose (its edge granularity can cost more tokens); the large token reductions XS3 achieves elsewhere come from re-representing *verbose* prose, not from compacting terse state. Second, the advantage requires the stack-and-dedup instruction; told to “rewrite concisely,” a model paraphrases XS3 too and drifts like prose.

B. Edge aging and generational compaction

Addressability enables policies that prose cannot express cleanly. Edges can be timestamped (@turn) and evicted by recency; critical invariants (DOIs, contracts) can be *pinned* and exempted; and clusters of stale edges can be reified into a single summary node that retains a pointer to what it replaced—a generational, GC-like compaction. Each of these is a deterministic operation on an edge set; none is

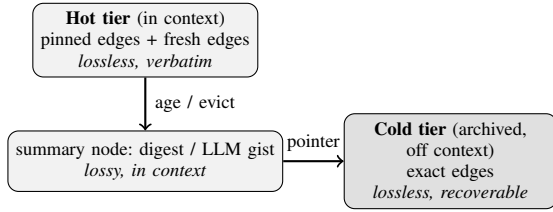


Fig. 3. Two-tier edge aging. Pinned and fresh edges stay verbatim in context; aged ephemera collapse to an in-context summary that points to their byte-exact edges in a cold archive, so the in-context state shrinks while nothing is lost.

expressible on prose without a rewrite, i.e., without risking the very corruption Table III measures. We implement a deterministic prototype combining recency eviction, pinning, and generational cluster-merge. Over a scripted twelve-turn session with five ephemera per turn it keeps every pinned fact—DOIs, invariants, commit hashes—*byte-exact*, bounds the live set to a fixed slot budget, and packs the sixty aged ephemera into four summary nodes; the compacted state is 73% smaller than the un-aged history (223 vs. 836 tokens) with zero loss of any pinned fact. The scheme is therefore *selectively lossy*: pinned facts are preserved byte-exact while ephemeral detail is reduced to a digest, and the 73% saving comes entirely from discarding that detail—this is eviction, not lossless compression. To keep the discarded detail *recoverable* rather than merely counted, we add a cold archive tier keyed by the summary node (Fig. 3): the exact edges are held off-context and returned byte-exact on demand, so the in-context saving costs no permanent loss. A summary node’s in-context text may be the deterministic digest or an LLM-generated gist of the sacrificed cluster; because that cluster is disposable by construction, a lossy gist there cannot corrupt any surviving fact. The contrast with prose is thus not that XS3 loses less but that it loses *safely*: what it keeps stays exact, what it drops is named, and the detail remains recoverable, whereas prose under the same budget silently corrupts what it appears to keep.

Recency alone, however, is blind to importance. On a probe where the durable facts were old and the disposable actions were fresh, a keep-the-freshest policy would retain *none* of the six critical facts, whereas an LLM asked to rank salience retained *all six*. This motivates a hybrid selection policy: the LLM *decides* what to demote—a ranking it does well—but a deterministic step *executes* the compaction, keeping survivors verbatim and letting the LLM summarize only the sacrificed cluster. True invariants are hard-pinned, so a mis-ranking can at worst pack a fact (recoverable via its pointer), never silently corrupt it. A full eviction policy and its evaluation across workloads remain future work.

VII. RELATED WORK

XS3 is closest in spirit to RDF and its reification, and to controlled natural languages such as Attempto Controlled English, but it inverts their usual priority: it optimizes for a pre-trained reader rather than a hand-written grammar, and it treats

the parser as secondary to spec-free comprehension. Its exactness mechanism resembles canonical serialization and graph canonical-labeling (e.g., Weisfeiler–Leman for blank nodes) rather than constrained decoding, which enforces grammar at generation time. Work on knowledge-graph compression (HDT, grammar-based methods) targets scale-driven redundancy; we deliberately do not claim token compression as XS3’s contribution, and our measurements show it is not one on information-dense input.

VIII. LIMITATIONS AND FUTURE WORK

XS3 trades raw logical expressiveness for a monotone-graph invariant: it has no native disjunction or unrestricted universal quantification, and negation is confined to a single edge. Auto-injecting XS3 formatting into a host’s automatic compaction is not currently possible in at least one widely used agent runtime, whose pre-compaction hook can only allow or block, not shape, the summary; the practical channel is an explicit, manual compaction instruction. Future work includes a formal eviction/aging policy, a study of multi-agent hand-off fidelity, and a larger, pre-registered comprehension benchmark across model families and sizes.

IX. CONCLUSION

XS3 is a notation in which one sentence is one graph edge, every document decodes deterministically to a triple graph, and the glyphs are chosen so a specification-free model reads them from prior knowledge alone. Its practical payoff is not brevity but *trust*: as accumulated agent state is compacted and re-read, atomic edges survive exactly or vanish visibly, whereas prose corrupts silently. We believe corruption-resistant, addressable state is the right foundation for machine-to-machine memory, and XS3 is a small, testable step toward it.

AVAILABILITY

The language card and the reference decoder are archived on Zenodo under concept DOI [10.5281/zenodo.21425813](https://doi.org/10.5281/zenodo.21425813), which always resolves to the latest version.

REFERENCES

- [1] G. Klyne and J. J. Carroll, “Resource Description Framework (RDF): Concepts and Abstract Syntax,” W3C Recommendation, 2004.
- [2] N. E. Fuchs, K. Kaljurand, and T. Kuhn, “Attempto Controlled English for Knowledge Representation,” in *Reasoning Web*, LNCS 5224, Springer, 2008, pp. 104–124.
- [3] J. D. Fernández, M. A. Martínez-Prieto, C. Gutiérrez, A. Polleres, and M. Arias, “Binary RDF Representation for Publication and Exchange (HDT),” *J. Web Semantics*, 2013.
- [4] B. Weisfeiler and A. Leman, “The reduction of a graph to canonical form and the algebra which appears therein,” *Nauchno-Tech. Informatsia*, ser. 2, no. 9, pp. 12–16, 1968.
- [5] L. Beurer-Kellner, M. Fischer, and M. Vechev, “Guiding LLMs the Right Way: Fast, Non-Invasive Constrained Generation,” in *Proc. ICML*, PMLR vol. 235, 2024.